Mitali Bhatia
D15C 16
MLDL

Experiment No 08

Aim: Design and train a Convolutional Neural Network (CNN) on image datasets

1. Dataset Source

The dataset used in this experiment is the **MNIST Handwritten Digit Dataset**.

- **Source:** Kaggle
- **Dataset Name:** MNIST Digit Recognizer
- **Link:** https://www.kaggle.com/datasets/oddrationale/mnist-in-csv

This dataset is widely used for image classification tasks and is a standard benchmark in deep learning.

---

2. Dataset Description

The MNIST dataset contains images of handwritten digits from 0 to 9.

- **Number of training instances:** 60,000
- **Number of testing instances:** 10,000
- **Image size:** 28 × 28 pixels
- **Number of features:** 784 (flattened image pixels)

Features:

- Each row represents an image
- Pixel values range from 0 to 255 (grayscale intensity)

Target Variable:

- Digit label (0 to 9)

Dataset Characteristics:

- Grayscale image dataset
- Multi-class classification problem (10 classes)
- Clean and structured dataset
- No missing values

---

3. Mathematical Formulation of the Algorithm

A Convolutional Neural Network (CNN) is used for image processing and classification.

Convolution Operation

$$\boxed{Feature\ Map = Input\ *\ Filter + Bias}$$

- A filter (kernel) slides over the image
- Extracts important features like edges and patterns

Activation Function (ReLU)

$$\boxed{f(x) = \max(0, x)}$$

- Removes negative values
- Helps model learn non-linear patterns

Pooling (Max Pooling)

$$\boxed{Output = \max(region)}$$

- Reduces image size
- Keeps important features

Fully Connected Layer

$$\boxed{y = Wx + b}$$

- Combines extracted features for classification

Softmax Function

$$\boxed{P(y_i) = \frac{e^{z_i}}{\sum e^{z_j}}}$$

- Converts outputs into probabilities for each class

Loss Function (Categorical Cross Entropy)

$$\boxed{Loss = -\sum y \log(p)}$$

---

4. Algorithm Limitations

- Requires large amount of data
- Training can be computationally expensive
- Needs GPU for faster training
- Model tuning can be complex
- Can overfit if too deep

---

5. Methodology / Workflow

5.1 Overall Workflow

1. Load MNIST dataset from CSV files
2. Separate features and labels
3. Normalize pixel values (0 to 1)
4. Reshape data into 28×28 images
5. Build CNN models
6. Train models using training data
7. Evaluate models on test data
8. Compare performance of different models
9. Plot accuracy graphs
10. Make predictions on test images

Flow:

Dataset → Preprocessing → Normalization → Reshaping

→ CNN Model → Training → Evaluation → Prediction → Visualization

---

5.2 CNN Workflow

1. Load training and testing datasets
2. Split input images (X) and labels (y)
3. Normalize pixel values by dividing by 255
4. Reshape data into 4D format (samples, height, width, channel)
5. Build CNN models:
    - Convolution layer

    - Pooling layer
    - Flatten layer
    - Dense layers
6. Compile model using Adam optimizer
7. Train model for multiple epochs
8. Evaluate accuracy on test data
9. Compare different model architectures (simple, standard, deep)
10. Display predictions on sample images

---

6. Performance Analysis

The model performance was evaluated using:

Accuracy

- Measures correct predictions out of total samples
- Accuracy is printed for each model (simple, standard, deep)

Training vs Validation Accuracy

- Shows learning performance over epochs
- Helps detect overfitting

Model Comparison

- Three models were tested:
    - Simple CNN
    - Standard CNN
    - Deep CNN

Interpretation

- Deep model generally performs better due to more layers
- Accuracy improves as model complexity increases
- Model successfully classifies handwritten digits

---

7. Hyperparameter Tuning

Different model configurations were tested:

Hyperparameters Used:

- Number of convolution layers
- Number of filters (32, 64)

- Number of epochs (5)
- Activation function (ReLU, Softmax)
- Optimizer (Adam)

Model Variants:

- **Simple Model:** 1 Conv layer
- **Standard Model:** 2 Conv layers
- **Deep Model:** Multiple Conv layers

Impact of Tuning:

- Increasing layers improved accuracy
- Better feature extraction in deeper models
- Improved classification performance

CODE:

```
# Import libraries

import pandas as pd

import numpy as np

import tensorflow as tf

from tensorflow import keras

from tensorflow.keras import layers

import matplotlib.pyplot as plt

# Load dataset (from uploaded CSV files)

train_data = pd.read_csv('mnist_train.csv')

test_data = pd.read_csv('mnist_test.csv')

# Split features and labels

x_train = train_data.iloc[:, 1:].values

y_train = train_data.iloc[:, 0].values

x_test = test_data.iloc[:, 1:].values
```

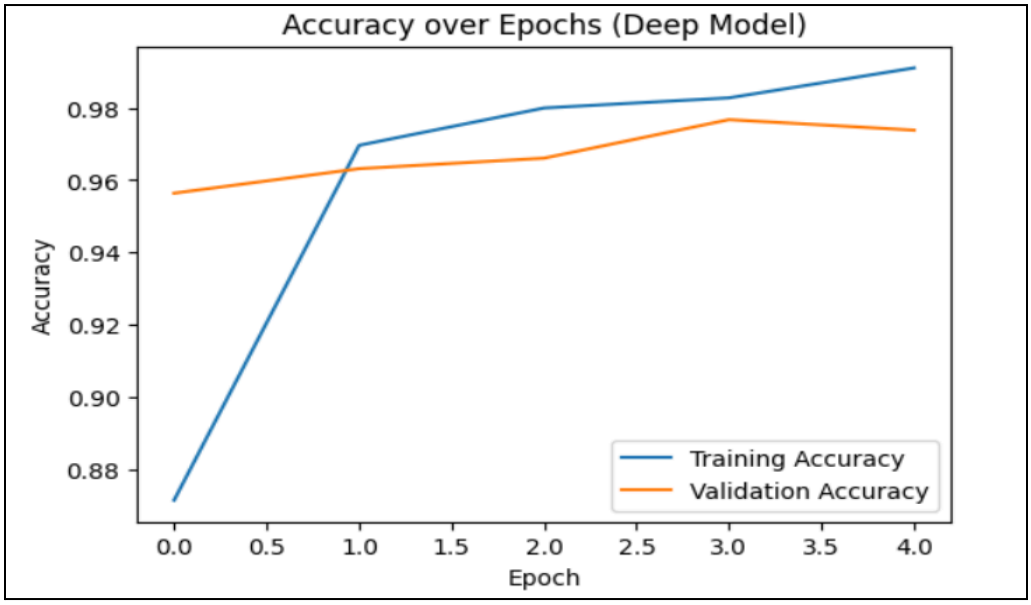
```
y_test = test_data.iloc[:, 0].values

# Normalize

x_train = x_train / 255.0

x_test = x_test / 255.0

# Reshape for CNN

x_train = x_train.reshape(-1,28,28,1)

x_test = x_test.reshape(-1,28,28,1)

print("\n--- Running CNN Training Trials ---")

# Function to build models

def build_model(type="simple"):

    model = keras.Sequential()

    if type == "simple":
model.add(layers.Conv2D(32,(3,3),activation='relu',input_shape=(28,28
,1)))

        model.add(layers.MaxPooling2D((2,2)))

    elif type == "standard":
model.add(layers.Conv2D(32,(3,3),activation='relu',input_shape=(28,28
,1)))

        model.add(layers.MaxPooling2D((2,2)))

        model.add(layers.Conv2D(64,(3,3),activation='relu'))

        model.add(layers.MaxPooling2D((2,2)))

    elif type == "deep":
model.add(layers.Conv2D(32,(3,3),activation='relu',input_shape=(28,28
,1)))

        model.add(layers.Conv2D(32,(3,3),activation='relu'))
```

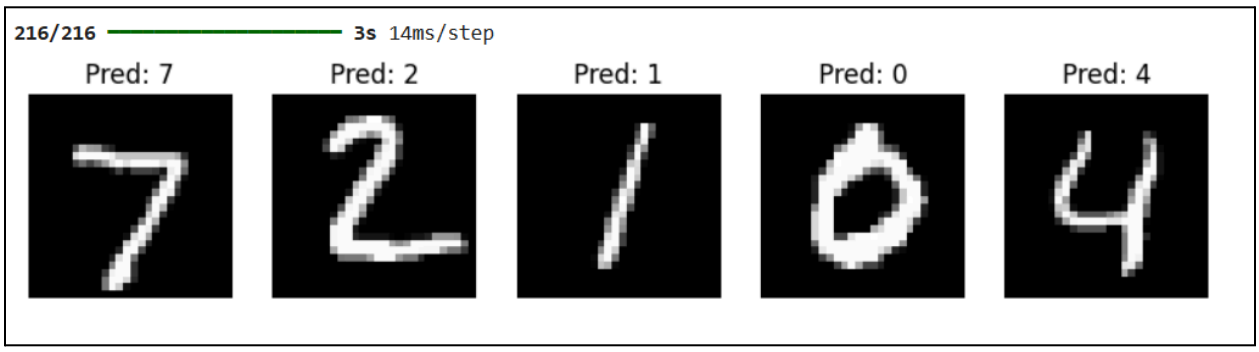
```
        model.add(layers.MaxPooling2D((2,2)))
        model.add(layers.Conv2D(64,(3,3),activation='relu'))
        model.add(layers.MaxPooling2D((2,2)))
    model.add(layers.Flatten())
    model.add(layers.Dense(64, activation='relu'))
    model.add(layers.Dense(10, activation='softmax'))
    model.compile(optimizer='adam',
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])
    return model
# Train models
histories = []
for name in ["simple","standard","deep"]:
    print(f"\nTraining Trial ({name.capitalize()})...")
    model = build_model(name)
    history = model.fit(x_train, y_train,
                        epochs=5,
                        validation_split=0.1,
                        verbose=0)
    loss, acc = model.evaluate(x_test, y_test, verbose=0)
    print(f"Trial ({name.capitalize()}) Test Accuracy: {acc:.4f}")
    histories.append((name, history, model))
```

```python
# Plot Accuracy Graph (Deep model)

name, history, model = histories[-1]

plt.figure(figsize=(6,4))

plt.plot(history.history['accuracy'], label='Training Accuracy')

plt.plot(history.history['val_accuracy'], label='Validation
Accuracy')

plt.xlabel('Epoch')

plt.ylabel('Accuracy')

plt.title(f'Accuracy over Epochs ({name.capitalize()} Model)')

plt.legend()

plt.show()

# Predictions

predictions = model.predict(x_test)

plt.figure(figsize=(10,2))

for i in range(5):

    plt.subplot(1,5,i+1)

    plt.imshow(x_test[i].reshape(28,28), cmap='gray')

    plt.title(f"Pred: {np.argmax(predictions[i])}")

    plt.axis('off')

plt.show()
```

OUTPUT:

```
Trial (Simple) Test Accuracy: 0.9629

Training Trial (Standard)...
Trial (Standard) Test Accuracy: 0.9734

Training Trial (Deep)...
Trial (Deep) Test Accuracy: 0.9776
```





Conclusion:

In this experiment, a Convolutional Neural Network was implemented to classify handwritten digits. Multiple CNN architectures were tested, and the results showed that deeper models provide better accuracy. The model successfully learned image features and performed accurate classification.